

第2章 递归与分治策略

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术；
 - (2) 大整数乘法、矩阵乘法；
 - (3) 棋盘覆盖；
 - (4) 合并排序和快速排序；
 - (5) 线性时间选择；
 - (6) 最接近点对问题 ；
 - (7) 循环赛日程表

2.3 二分搜索技术

给定已按**升序排好序**的 n 个元素 $a[0:n-1]$, 现要在这 n 个元素中找出一特定元素 x 。

分析:

BinarySearch (a, x)
在有序数组 a 中查询 x

数组 a

7	12	30	35	75	83	87	90	97	99
---	----	----	----	----	----	----	----	----	----

x 75

顺序查找算法的时间复杂度是 $O(?)$

2.3 二分搜索技术

给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。

分析：

该问题的规模缩小到一定的程度就可以容易地解决；

该问题可以分解为若干个规模较小的相同问题；

分解出的子问题的解可以合并为原问题的解；

分解出的各个子问题是相互独立的。

分析：很显然此问题分解出的子问题相互独立，即在 $a[i]$ 的前面或后面查找 x 是独立的子问题，因此满足分治法的第四个适用条件。

2.3 二分搜索技术

给定**已按升序排好序**的 n 个元素 $a[0:n-1]$, 在这 n 个元素中找出一特定元素 x 。

```
template<class Type>
int BinarySearch( a[], x,l,r)
{
    int m = (l+r)/2;
    if (x == a[m]) return m;
    else if (x < a[m])
        BinarySearch(a, x, l, m-1);
    else BinarySearch(a, x, m+1,r);
}
return -1;
```

递推方程:

$$T(n) = T(n/2) + c$$

复杂性分析?

2.3 二分搜索技术

主定理

复杂性分析:

迭代法

$$T(n) = T(n/2) + c$$

$$= T(n/4) + c + c$$

$$= T(n/8) + c + c + c$$

$$= T(n/2^k) + kc \quad \text{设 } k = \log n$$

$$= T(1) + c \log n$$

$$= O(\log n)$$

$$T(n) = a T(n/b) + f(n)$$

$$T(n) = T(n/2) + c$$

$$a=1, b=2, d=0$$

$$n^{\log_b a} = 1; \quad n^d = 1$$

$$T(n) = O(\log n)$$

常用的递推式和复杂度

递推式	复杂度
$T(n) = T(n/2) + O(1)$	$O(\log n)$
$T(n) = T(n - 1) + O(1)$	$O(n)$
$T(n) = T(n/2) + O(n)$	$O(n)$
$T(n) = 2 * T(n/2) + O(1)$	$O(n)$
$T(n) = 2 * T(n/2) + O(n)$	$O(n \log n)$
$T(n) = T(n - 1) + O(n)$	$O(n^2)$
$T(n) = 2 * T(n - 1) + O(1)$	$O(2^n)$
$T(n) = 2 * T(n - 1) + O(n)$	$O(2^n)$

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术;
 - (2) 大整数乘法、矩阵乘法;
 - (3) 棋盘覆盖;
 - (4) 合并排序和快速排序;
 - (5) 线性时间选择;
 - (6) 最接近点对问题 ;
 - (7) 循环赛日程表

2.4 大整数的乘法

设计一个有效的算法，可以进行两个n位二进制大整数的乘法运算

小学的方法

$$\begin{array}{r} 111 \\ 1011 \\ \hline 111 \\ 111 \\ 000 \\ 111 \\ \hline 1001101 \end{array}$$

复杂度: $O(n^2)$

✗效率太低

2.4 大整数的乘法

设计算法，可以进行两个n位二进制大整数的乘法运算

小学的方法： $O(n^2)$ ✖效率太低

分治法：

$$X = \begin{array}{|c|c|} \hline a & b \\ \hline \end{array} = a 2^{n/2} + b$$

$$Y = \begin{array}{|c|c|} \hline c & d \\ \hline \end{array} = c 2^{n/2} + d$$

$$XY = ac 2^n + (ad+bc) 2^{n/2} + bd$$

2.4 大整数的乘法

设计算法，可以进行两个n位二进制大整数的乘法运算

小学的方法： $O(n^2)$ ✖效率太低

分治法：

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

X =

Y =

$$T(n) = O(n^2) \quad \text{✖没有改进}$$

b
d

$$XY = ac \cdot 2^n + (ad+bc) \cdot 2^{n/2} + bd$$

2.4 大整数的乘法

请设计一个有效的算法，可以进行两个n位大整数的乘法运算

小学的方法： $O(n^2)$ ✖效率太低

分治法：

$$XY = ac 2^n + (ad+bc) 2^{n/2} + bd \quad \text{✖效率太低}$$

为了降低时间复杂度，必须减少乘法的次数。

1960 年，数学家阿纳托利·卡拉苏巴 (Anatoly Karatsuba) 设计的算法

$$XY = ac 2^n + ((a-b)(d-c) + ac + bd) 2^{n/2} + bd$$

2.4 大整数的乘法

请设计一个有效的算法，可以进行两个n位大整数的乘法运算

小学的方

分治法:

$XY =$

为

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

$$T(n) = O(n^{\log_3 3}) = O(n^{1.59}) \checkmark \text{较大的改进}$$

1960 年，数学家阿纳托利·卡拉苏巴 (Anatoly Karatsuba) 设计的算法

$$XY = \underline{ac} \ 2^n + ((\underline{a-b})(\underline{d-c}) + \underline{ac} + \underline{bd}) \ 2^{n/2} + \underline{bd}$$

2.4 大整数的乘法

请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

小学的方法: $O(n^2)$ ✖ 效率太低

Karatsuba分治法: $O(n^{1.59})$ ✔ 较大的改进

更快的方法??

如果将大整数分成更多段，将有可能得到更优的算法?

分3段?

2.4 大整数的乘法

如果将大整数分成更多段，分3段？

$$X = A \times 2^{2n/3} + B \times 2^{n/3} + C$$

$$Y = D \times 2^{2n/3} + E \times 2^{n/3} + F$$

$$X \times Y = 2^{4n/3} \times AD + 2^{3n/3} \times (AE + BD) + 2^{2n/3} \times (AF + CD + BE) + 2^{n/3} \times (BF + CE) + CF$$

其中AD、AE、BD、AF、CD、BE、BF、CE、CF共9个乘积项

$$T(n) = 5T(n/3) + O(n)$$

$$O(n^{1.46})$$

Toom-Cook-3算法

$$M_1 = A * D$$

$$M_2 = C * F$$

$$M_3 = (A + B + C) * (D + E + F)$$

$$M_4 = (A - B + C) * (D - E + F)$$

$$M_5 = (A - 2B + 4C) * (D - 2E + 4F)$$

5次

$$AE + BD = (3M_1 - 12M_2 + 2M_3 - 6M_4 + M_5) / 6$$

$$AF + CD + BE = (M_3 + M_4) / 2 - M_1 - M_2$$

$$CE + BF = (-3M_1 + 12M_2 + M_3 + 3M_4 - M_5) / 6$$

可以看出，分割越大，时间复杂度就越低，但是所要计算的中间项以及合并最终结果的过程就会越复杂，开销会增加

2.4 大整数的乘法

请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

小学的方法: $O(n^2)$

✗效率太低

分治法: $O(n^{1.59})$

✓较大的改进

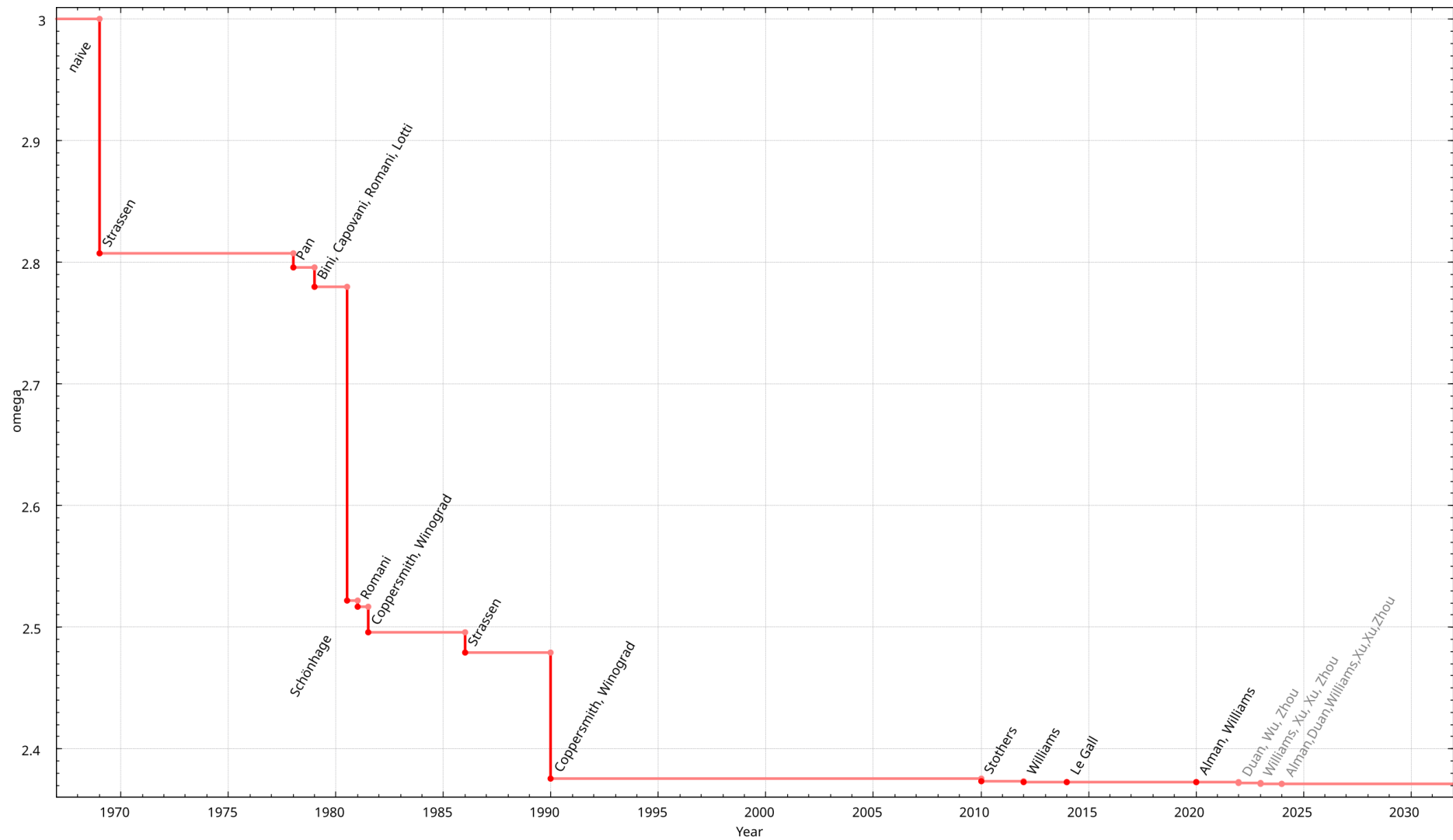
更快的方法??

如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。

2019年的一项研究成果，采用了1729维的快速傅利叶变换(FFT)使得计算速度达到了 $O(n \log n)$ 。该方法也可以看作是一个复杂的分治算法。

Integer multiplication in time $O(n \log n)$

David Harvey, Joris Van Der Hoeven



2.5 矩阵乘法

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

$$1 \times 5 + 2 \times 7 = 19$$

$$1 \times 6 + 2 \times 8 = 22$$

$$3 \times 5 + 4 \times 7 = 43$$

$$3 \times 6 + 4 \times 8 = 50$$

1. 朴素算法（三重循环）

- 原理：直接按照矩阵乘法的定义计算每个元素。

python

```
def naive_multiply(A, B):  
    n = len(A)  
    C = [[0]*n for _ in range(n)]  
    for i in range(n):  
        for j in range(n):  
            for k in range(n):  
                C[i][j] += A[i][k] * B[k][j]  
    return C
```

- 时间复杂度： $O(n^3)$
- 优缺点：
 - 优点：实现简单，无额外内存开销。
 - 缺点：效率低，无法处理大规模矩阵。

矩阵乘法的分治实现基于将大矩阵分解为较小的子矩阵，递归计算子矩阵的乘积，再合并结果。以下是分治法实现矩阵乘法的详细步骤：

1. 分治策略

假设两个 $n \times n$ 矩阵 A 和 B 相乘，且 n 是 2 的幂次（若否，可通过填充 0 调整）。将 A 和 B 各自划分为 4 个大小相等的子矩阵：

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

结果矩阵 $C = A \times B$ 同样划分为 4 个子矩阵：

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

根据矩阵乘法规则，每个子矩阵的计算为：

$$C_{11} = A_{11}B_{11} + A_{12}B_{21},$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22},$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21},$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}.$$

2. 递归过程

- 分解**：将 A 和 B 分解为子矩阵。
- 递归计算**：对每个子矩阵乘积进行递归计算。
- 合并**：将子矩阵乘积的结果合并为最终结果。

时间复杂度

- 递归式**： $T(n) = 8T(n/2) + O(n^2)$ （每次分解为 8 次子矩阵乘法和 $O(n^2)$ 的加法）。
- 主定理**：解得 $T(n) = O(n^3)$ ，与朴素算法相同，但为后续优化（如 **Strassen 算法**）提供基础。

2.5 Strassen矩阵乘法

1969年

Strassen 算法是一种基于分治法的矩阵乘法优化算法，通过减少递归调用的次数，将时间复杂度从传统分治法的 $O(n^3)$ 降低到 $O(n^{\log_2 7}) \approx O(n^{2.81})$ 。其核心思想是通过巧妙的数学变换，将矩阵乘法中的 **8 次子矩阵乘法** 减少为 **7 次**，从而优化计算效率。

1. 算法核心思想

假设矩阵 A 和 B 是 $2^k \times 2^k$ 的方阵（否则填充 0 对齐），将其划分为四个子矩阵：

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

传统分治法需要计算 8 次子矩阵乘法（如 $A_{11}B_{11}$, $A_{12}B_{21}$ 等）。

Strassen 的突破：通过构造 7 个中间矩阵 $M_1, M_2, \dots, M_7$ ，仅需 7 次子矩阵乘法和若干次加减法，即可组合出结果矩阵 $C = A \times B$ 。

2. 具体步骤

(1) 定义 7 个中间矩阵

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

(2) 合并结果

通过中间矩阵计算 C 的四个子块：

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

2.5 Strassen矩阵乘法

4. 时间复杂度分析

- 递归关系式:

$$T(n) = 7T(n/2) + O(n^2)$$

每次递归将问题分解为 7 个子问题（而非传统分治法的 8 个），外加 $O(n^2)$ 的加减法操作。

- 主定理求解:

$a = 7, b = 2, \log_b a = \log_2 7 \approx 2.81$, 因此时间复杂度为 $O(n^{\log_2 7}) \approx O(n^{2.81})$ 。

$$O(n^{\log_2 7})$$

2.5 Strassen矩阵乘法

5. 优缺点

优点:

- 理论时间复杂度低于朴素分治法。
- 是首个突破 $O(n^3)$ 的矩阵乘法算法，启发了后续更优算法（如 Coppersmith-Winograd 算法）。

缺点:

- 常数因子较大：由于递归的额外开销和复杂的加减法操作，实际应用中当矩阵规模较小时，效率可能不如朴素算法。
- 数值稳定性问题：多次加减法可能导致浮点运算误差累积。

6. 实际应用

- 大矩阵运算：当矩阵规模极大时（如 $n > 1000$ ），Strassen 算法开始显现优势。
- 并行计算：分治特性适合分布式计算框架（如 MapReduce）。
- 算法理论：证明了矩阵乘法的时间复杂度可以低于 $O(n^3)$ ，推动了理论计算机科学的发展。

2.5 Strassen矩阵乘法

算法	时间复杂度	常数因子 (实际)	适用场景
传统矩阵乘法	$O(n^3)$	1 (基准)	小矩阵 ($n < 1000$)
Strassen算法 (纯)	$O(n^{2.807})$	20–50	大矩阵 ($n \geq 10^4$)
Strassen算法 (混合)	$O(n^{2.807})$	5–15	中大规模矩阵 ($n \geq 2000$)

4. 性能示例

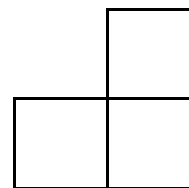
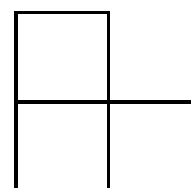
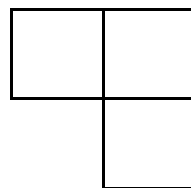
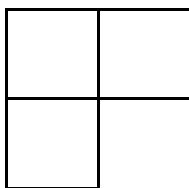
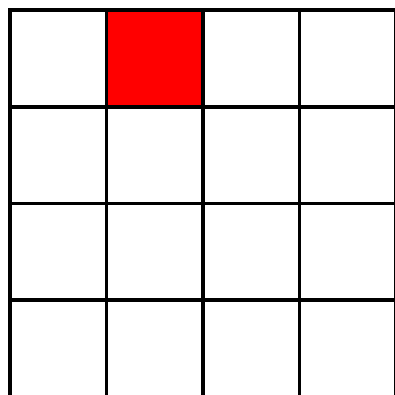
- 当 $n = 10^4$:
 - 传统算法: 约 10^{12} 次操作 (假设常数因子为1)。
 - Strassen混合实现: 约 $15 \cdot (10^4)^{2.807} \approx 15 \cdot 10^{11.22} \approx 2.4 \times 10^{12}$ 次操作。
 - **加速比**: $10^{12} / 2.4 \times 10^{12} \approx 0.42$ (即传统算法更快)。
 - **结论**: 此时Strassen算法因常数因子过大, 反而更慢。

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术;
 - (2) 大整数乘法;
 - (3) 棋盘覆盖;
 - (4) 合并排序和快速排序;
 - (5) 线性时间选择;
 - (6) 最接近点对问题 ;
 - (7) 循环赛日程表

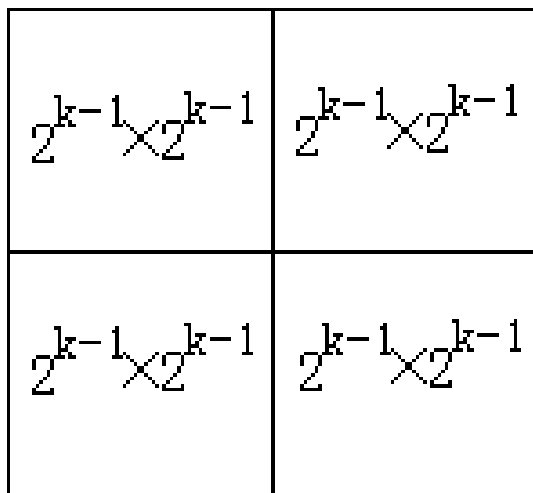
2.6 棋盘覆盖

在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘。在**棋盘覆盖问题**中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。

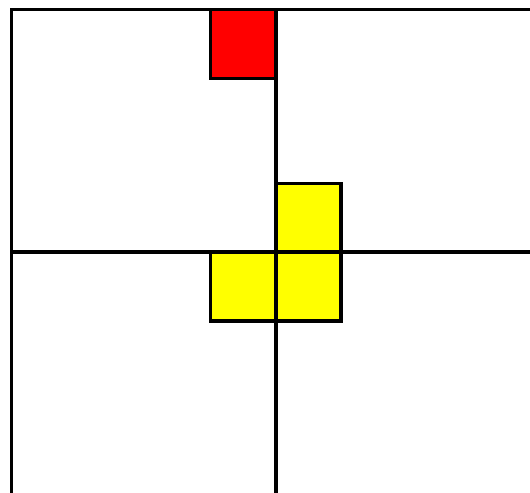


2.6 棋盘覆盖

当 $k > 0$ 时，将 $2^k \times 2^k$ 棋盘**分割**为4个 $2^{k-1} \times 2^{k-1}$ 子棋盘(a)所示.特殊方格必位于4个较小子棋盘之一中，其余3个子棋盘中无特殊方格。为了将这3个无特殊方格的子棋盘转化为特殊棋盘，可以用一个L型骨牌覆盖这3个较小棋盘的会合处，如(b)所示，从而将原问题转化为4个较小规模的棋盘覆盖问题。递归地使用这种分割，直至棋盘简化为棋盘 1×1 。



(a)



(b)

棋盘覆盖问题中数据结构的设计：

1. **棋盘**：可以用一个二维数组`board[size][size]`表示一个棋盘，其中， $\text{size}=2^k$ 。为了在递归处理的过程中使用同一个棋盘，将数组`board`设为全局变量；
2. **子棋盘**：`board[size][size]`表示整个棋盘，子棋盘由棋盘左上角的下标tr、tc和棋盘大小s表示；
3. **特殊方格**：用`board[dr][dc]`表示特殊方格，`dr`和`dc`是该特殊方格在二维数组`board`中的下标；
4. **L型骨牌**：一个 $2^k \times 2^k$ 的棋盘中有有一个特殊方格，所以，用到L型骨牌的个数为 $(4^k - 1)/3$ ，将所有L型骨牌从1开始连续编号，用一个全局变量`t`表示。

2.6 棋盘覆盖

```
void chessBoard(int tr, int tc, int dr, int dc, int size)
{
    if (size == 1) return;
    int t = tile++; // L型骨牌号 s = size/2; // 分割棋盘
    // 覆盖左上角子棋盘
    if (dr < tr + s && dc < tc + s)
        // 特殊方格在此棋盘中
        chessBoard(tr, tc, dr, dc, s);
    else { // 此棋盘中无特殊方格
        // 用 t 号L型骨牌覆盖右下角
        board[tr + s - 1][tc + s - 1] = t;
        // 覆盖其余方格
        chessBoard(tr, tc, tr+s-1, tc+s-1, s);
    }
    // 覆盖右上角子棋盘
    if (dr < tr + s && dc >= tc + s)
        // 特殊方格在此棋盘中
        chessBoard(tr, tc+s, dr, dc, s);
    else { // 此棋盘中无特殊方格
        // 用 t 号L型骨牌覆盖左下角
```

```
        board[tr + s - 1][tc + s] = t;
        // 覆盖其余方格
        chessBoard(tr, tc+s, tr+s-1, tc+s, s);
    }
    // 覆盖左下角子棋盘
    if (dr >= tr + s && dc < tc + s)
        // 特殊方格在此棋盘中
        chessBoard(tr+s, tc, dr, dc, s);
    else { // 用 t 号L型骨牌覆盖右上角
        board[tr + s][tc + s - 1] = t;
        // 覆盖其余方格
        chessBoard(tr+s, tc, tr+s, tc+s-1, s);
    }
    // 覆盖右下角子棋盘
    if (dr >= tr + s && dc >= tc + s)
        // 特殊方格在此棋盘中
        chessBoard(tr+s, tc+s, dr, dc, s);
    else { // 用 t 号L型骨牌覆盖左上角
        board[tr + s][tc + s] = t;
        // 覆盖其余方格
        chessBoard(tr+s, tc+s, tr+s, tc+s, s);
    }
}
```

2.6 棋盘覆盖

复杂度分析

设 $T(k)$ 是算法覆盖一个 $2^k \times 2^k$ 棋盘需要的时间，满足以下递归方程：

$$T(k) = \begin{cases} O(1) & k = 0 \\ 4T(k-1) + O(1) & k > 0 \end{cases}$$

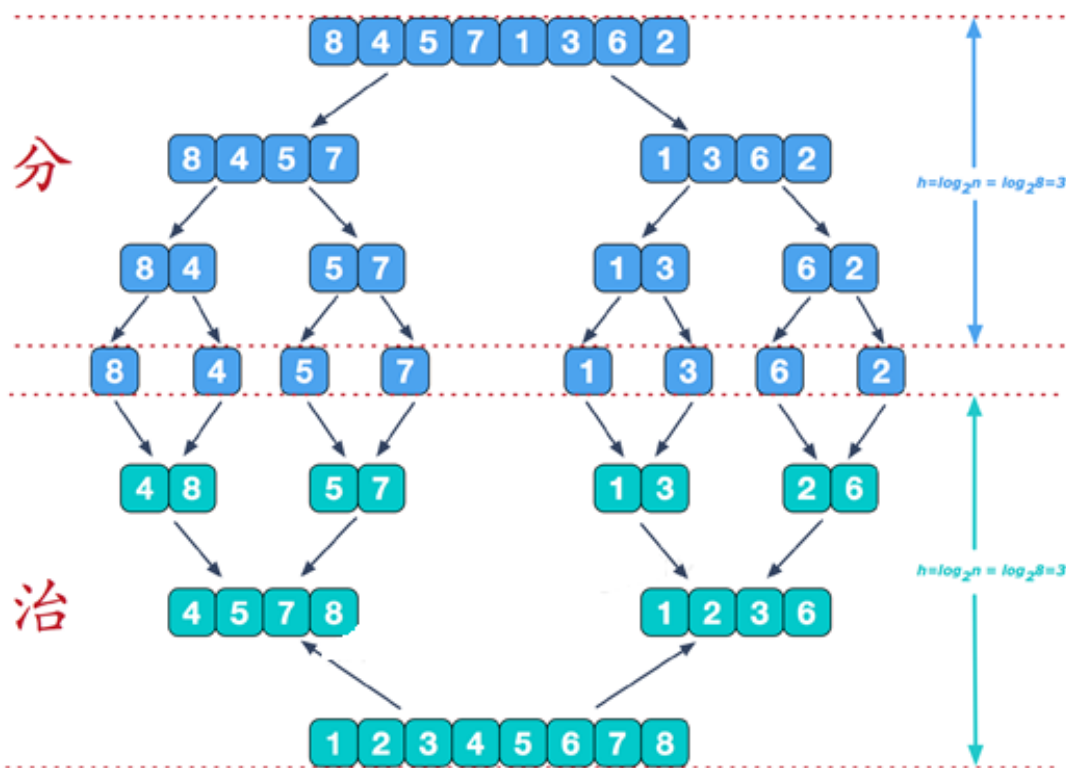
$$T(k) = O(4^k)$$

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术;
 - (2) 大整数乘法;
 - (3) 棋盘覆盖;
 - (4) 合并排序和快速排序;
 - (5) 线性时间选择;
 - (6) 最接近点对问题 ;
 - (7) 循环赛日程表

2.7 合并排序

合并排序法采用分治策略实现排序，把待排序元素分成大小大致相同的两个子集分别排序，然后再把有序子序列合并为整体有序序列。

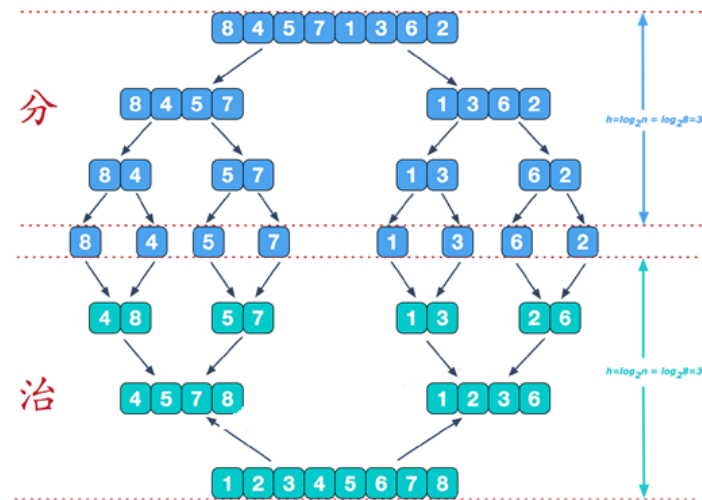


2.7 合并排序

基本思想：将待排序元素分成大小大致相同的2个子集合，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。

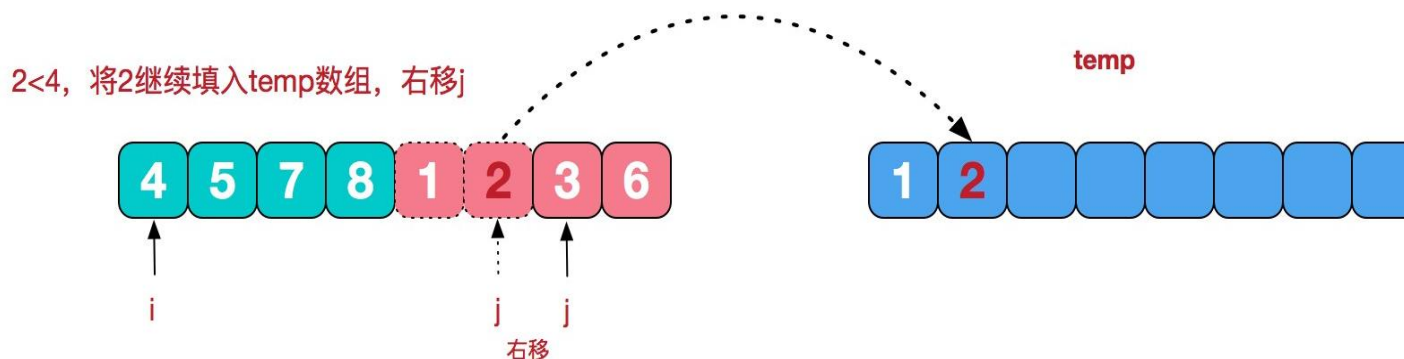
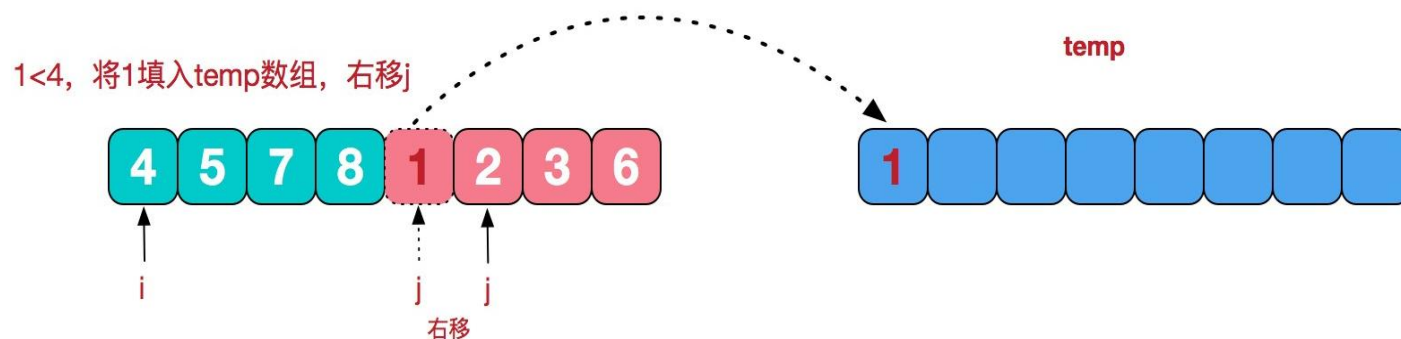
```
void MergeSort(Type a[], int left, int right)
```

```
{  
    if (left < right) {  
        int i = (left + right) / 2; //取中点  
        mergeSort(a, left, i);  
        mergeSort(a, i + 1, right);  
        merge(a, b, left, i, right); //合并到数组b  
        copy(a, b, left, right);    //复制回数组a  
    }  
}
```

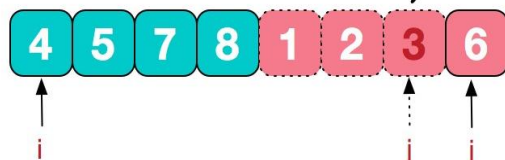


merge算法：对两个有序数组的合并。

1. 初始状态下，两个指针分别指向待合并数组的第一个元素。
2. 比较这两个元素的大小，将较小的元素添加到新创建的数组中；
3. 被复制数组中的指针后移，指向较小元素的后继元素。
4. 上述操作一直持续到两个数组中的一个被处理完为止。
5. 在未处理完的数组中，剩下的元素被复制到新创建数组的尾部。

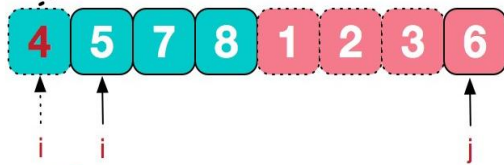


3<4, 将3填入temp数组, 右移j



右移

4<6, 此时将4填入temp数组, 右移i



右移

merge

继续重复这种比较+填入的步骤, 直到右子序列已经填完, 这时将左边剩余的7和8依次填入



temp

$O(n)$

最后, 将temp中的内容全部拷到原数组中去, 排序完成



temp

copy

copy

2.7 合并排序

基本思想：将待排序元素分成大小大致相同的2个子集合，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。

void MergeSort(Type a[], int left, int right)

复杂度分析

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

$T(n)=O(n\log n)$ 渐进意义下的**最优**算法

merge(a, b, left, i, right); //合并到数组b

copy(a, b, left, right); //复制回数组a

}

}

2.7 合并排序

📖 最坏时间复杂度: $O(n \log n)$

📖 平均时间复杂度: $O(n \log n)$

合并排序
优点: 稳定
数十分接近
论上能够过

缺点: 算法

排序算法	最好情况下时间复杂度	平均情况下时间复杂度	最坏情况下时间复杂度	适用场景
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	教学示例、小规模数据或几乎有序的数据。
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	教学示例、小规模数据或对交换次数有严格限制的场景。
插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	小规模数据、几乎有序的数据或作为其他排序算法的子过程（如快速排序优化）。
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	大规模数据、需要稳定排序的场景、外部排序（如磁盘文件排序）。
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	大规模数据、对内存使用有限制的场景（原地排序）。
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	大规模数据、需要原地排序且对最坏情况时间复杂度有要求的场景。

欠里

2.8 快速排序

相同之处： 分治

区别之处：

合并排序：将问题划分成两个子问题很快，
算法主要工作在于合并子问题的解

快速排序：主要工作在于划分节点，而不需
要再去合并子问题的解

2.8 快速排序

合并排序：按照元素在数组的位置进行划分

快速排序：按照(基准)元素的值进行划分

Partition

$$\underbrace{A[0] \dots A[s-1]}_{\text{都} \leq A[s]} \quad A[s] \quad \underbrace{A[s+1] \dots A[n-1]}_{\text{都} \geq A[s]}$$

$A[s]$ 已经位于有序数组中的最终位置，对 $A[s]$ 前后子数组进行排序

2.8 快速排序

```
template<class Type>
void QuickSort (Type a[], int p, int r)
{
    if (p<r) {
        int q=Partition(a,p,r); //q是基准元素所在的位置
        QuickSort (a,p,q-1); //对左半段排序
        QuickSort (a,q+1,r); //对右半段排序
    }
}
```

2.8 快速排序

```
template<class Type>
```

```
int Partition (Type a[], int p, int r)
```

```
int i = p, j = r + 1;
```

Type $x=a[p];$

```
// 将< x的元素交换到左边区域
```

```
// 将> x的元素交换到右边区域
```

```
while (true) {
```

```
while (a[++i] < x);
```

```
while (a[-j] > x);
```

```
if (i >= j) break;
```

```
swap(a[i], a[j]);
```

}

```
a[p] = a[j];
```

```
a[j] = x;
```

```
return j;
```

}

在快速排序中，记录的比较和交换是从两端向中间进行的，值较大的记录一次就能交换到后面单元，较小的记录一次就能交换到前面单元。

27	99	0	8	13	64	86	16	7	10	88	25	90
	<i>i</i>										<i>j</i>	
27	25	0	8	13	64	86	16	7	10	88	99	90
					<i>i</i>				<i>j</i>			
27	25	0	8	13	10	86	16	7	64	88	99	90
					<i>i</i>			<i>j</i>				
27	25	0	8	13	10	7	16	86	64	88	99	90
						<i>j</i>		<i>i</i>				
16	25	0	8	13	10	7	27	86	64	88	99	90

Partition的时间复杂度是 $O(n)$

2.8 快速排序

复杂性分析

快速排序的运行时间与划分是否对称有关

最好时间复杂度：如果所有的分裂点位于相应子数组的中点，这就是最优的情况

$$T(n) = 2T(n/2) + O(n) \quad O(n \log n)$$

最坏时间复杂度：所有的分裂点都趋于极端：两个子数组有一个为空，而另一个子数组仅仅比被划分的数组少一个元素。

$$T(n) = T(n-1) + O(n) \quad O(n^2)$$

平均时间复杂度 $O(n \log n)$

2.8 快速排序

快速排序算法的性能取决于划分的对称性。通过修改算法**Partition**，可以设计出**采用随机选择策略的快速排序算法**。在快速排序算法的每一步中，当数组还没有被划分时，可以在 $a[p:r]$ 中随机选出一个元素作为划分基准，这样可以使划分基准的选择是随机的，从而可以**期望划分是较对称的**。

```
template<class Type>
int RandomizedPartition (Type a[], int p, int r)
{
    int i = Random(p,r);
    Swap(a[i], a[p]);
    return Partition (a, p, r);
}
```

2.8 快速排序

```
template<class Type>
```

```
void RandomizeQuickSort (Type a[], int p, int r)
```

```
{ 最坏时间复杂度:  $O(n^2)$ 
```

```
  平均时间复杂度:  $O(n\log n)$ 
```

```
    int q=RandomizePartition(a,p,r);
```

```
    RandomizeQuickSort (a,p,q-1); //对左半段排序
```

```
    RandomizeQuickSort (a,q+1,r); //对右半段排序
```

```
}
```

```
}
```

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术;
 - (2) 大整数乘法;
 - (3) 棋盘覆盖;
 - (4) 合并排序和快速排序;
 - (5) 线性时间选择;
 - (6) 最接近点对问题 ;
 - (7) 循环赛日程表

2.9 线性时间选择

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素?

- $k = 1$: 最小值;
- $k = n$: 最大值;
- $k = n/2$: 中位数

最简单的方法: 排序+选择

$$T(n) = \Theta(n \log n) + \Theta(1) = \Theta(n \log n)$$

有没有可能更简单?

2.8 快速排序

```
template<class Type>
```

```
int Partition (Type a[], int p, int r)
```

 $\{$

```
int i = p, j = r + 1;
```

Type $x=a[p];$

```
// 将< x的元素交换到左边区域
```

```
// 将> x的元素交换到右边区域
```

```
while (true) {
```

```
while (a[++i] < x);
```

```
while (a[- j] >x);
```

```
if (i >= j) break;
```

```
swap(a[i], a[j]);
```

}

```
a[p] = a[j];
```

```
a[j] = x;
```

```
return j;
```

}

在快速排序中，记录的比较和交换是从两端向中间进行的，值较大的记录一次就能交换到后面单元，较小的记录一次就能交换到前面单元。

27	99 <i>i</i>	0	8	13	64	86	16	7	10	88	25 <i>j</i>	90
27	25	0	8	13	64 <i>i</i>	86	16	7	10 <i>j</i>	88	99	90
27	25	0	8	13	10	86 <i>i</i>	16	7 <i>j</i>	64	88	99	90
27	25	0	8	13	10	7	16 <i>j</i>	86 <i>i</i>	64	88	99	90
16	25	0	8	13	10	7	27	86	64	88	99	90

Partition的时间复杂度是 $O(n)$

2.9 线性时间选择

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素

```
template<class Type>
```

```
Type RandomizedSelect(Type a[],int p,int r,int k)
```

```
{  
    if (p==r) return a[p];  
    int i=RandomizedPartition(a,p,r), j=i-p+1;  
    if (k<=i) return RandomizedSelect(a, p, i, k);
```

在最坏情况下, 算法**RandomizedSelect**需要 $O(n^2)$ 计算时间
但可以证明, 算法**RandomizedSelect**可以在 $O(n)$ 平均时间内
找出 n 个输入元素中的第 k 小元素。

有没有最坏情况下依然为线性的算法?

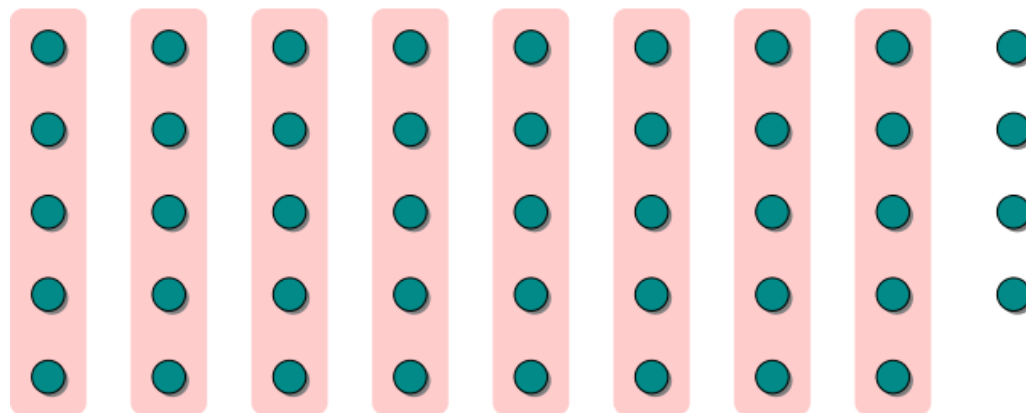
2.9 线性时间选择

计算耗时分析

$T(n)$	SELECT(i, n)
$\Theta(n)$	{ 1. Divide the n elements into groups of 5. Find the median of each 5-element group by rote.
$T(n/5)$	{ 2. Recursively SELECT the median x of the $\lfloor n/5 \rfloor$ group medians to be the pivot.
$\Theta(n)$	3. Partition around the pivot x . Let $k = \text{rank}(x)$.
$T(3n/4)$	{ 4. if $i = k$ then return x elseif $i < k$ then recursively SELECT the i th smallest element in the lower part else recursively SELECT the $(i-k)$ th smallest element in the upper part

2.9 线性时间选择

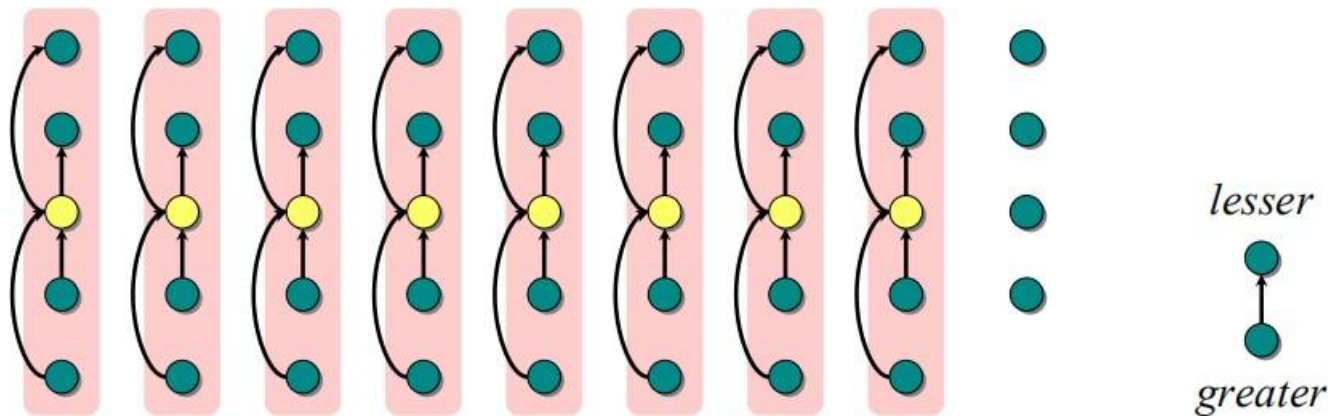
Select算法:



1) 将 n 个输入元素划分成 $\lfloor n/5 \rfloor$ 个组，每组5个元素。剩下 $n \bmod 5$ 个元素

2.9 线性时间选择

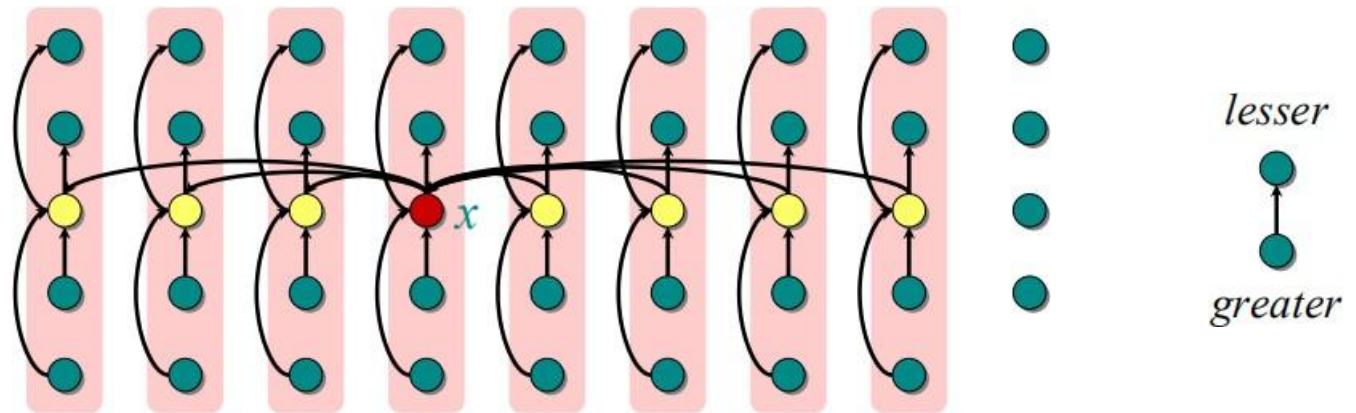
Select算法:



用排序算法，将每组中的元素排好序，取出每组的中位数，共 $\lfloor n/5 \rfloor$ 个。

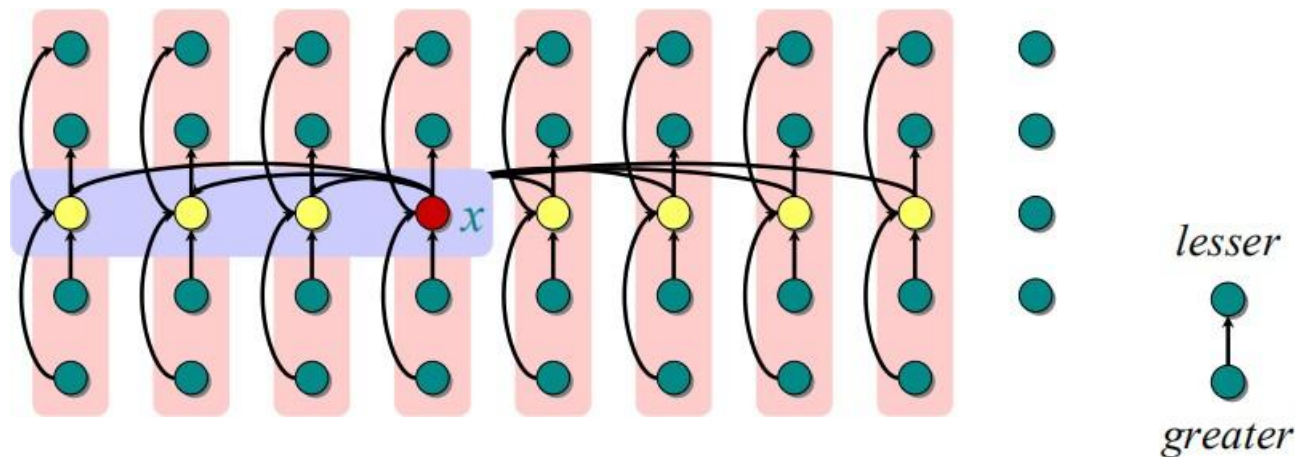
2.9 线性时间选择

Select算法:



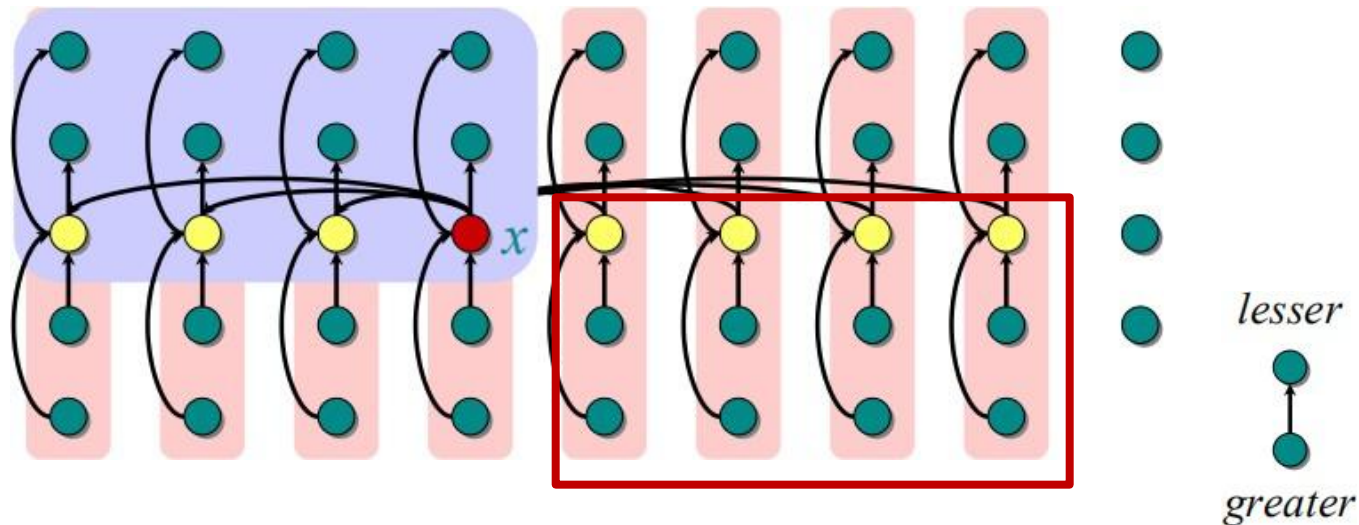
- 2) 调用**Select**来找出这 $\lfloor n/5 \rfloor$ 个中位数的中位数 x 。

2.9 线性时间选择



最少有一半中位数 $\leq x$, 也就是
 $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$

2.9 线性时间选择

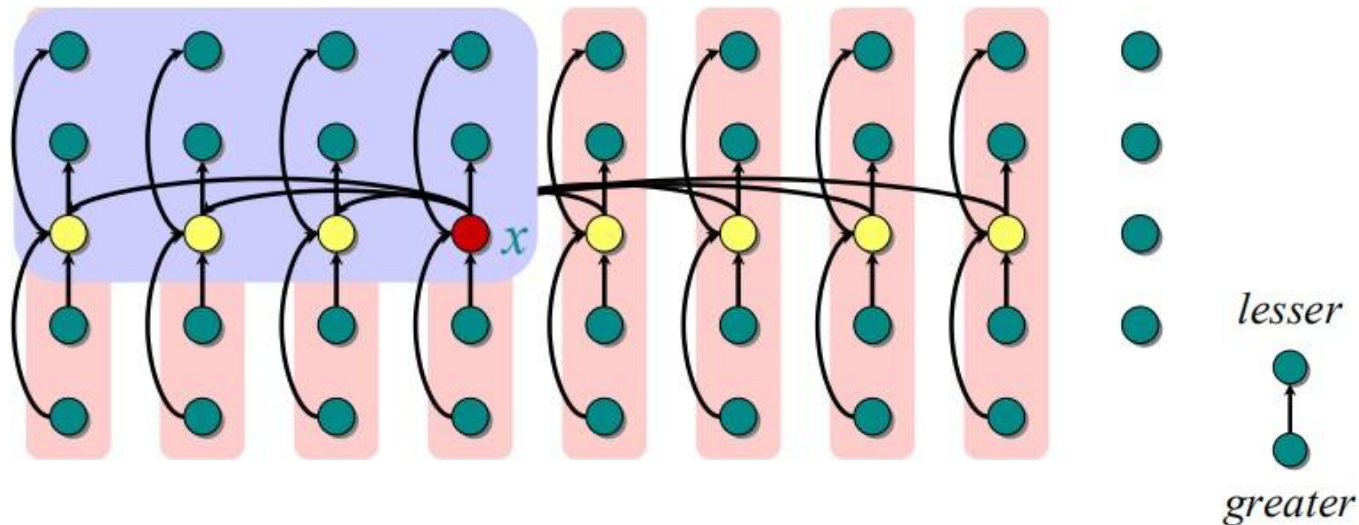


因此, $3 \lfloor (n-5)/10 \rfloor$ 个数 $\leq x$

同理, $3 \lfloor (n-5)/10 \rfloor$ 个数 $\geq x$

选取 x 为划分基准的优点?

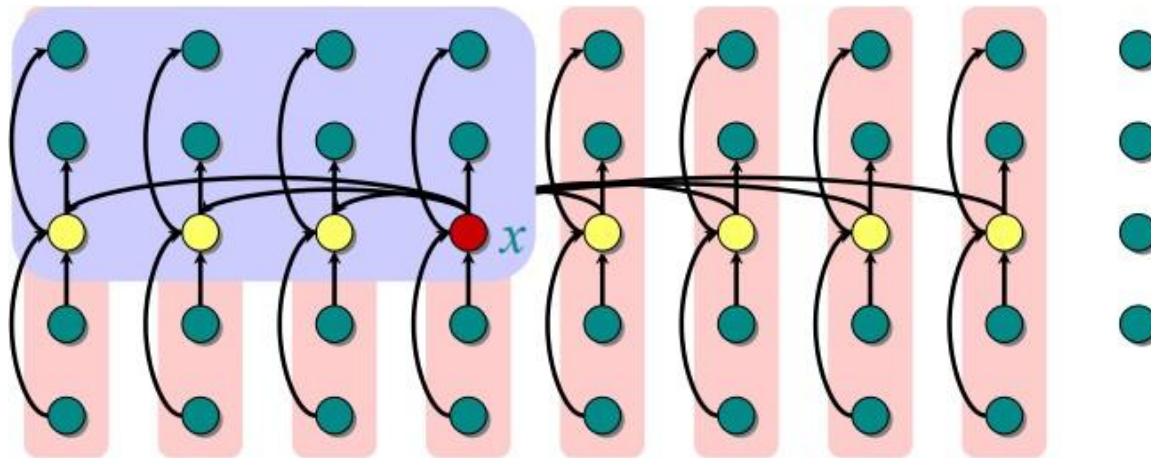
2.9 线性时间选择



因此, $3 \lfloor (n-5)/10 \rfloor$ 个数 $\leq x$

- 对于 $n \geq 75$, 有 $3 \lfloor (n-5)/10 \rfloor \geq n/4$ 。
- 所以, 按此基准划分所得的两个子数组长度至少缩短 $1/4$

2.9 线性时间选择



4) 根据找到的 x ，对全部 n 个元素递归划分

对于 $n \geq 75$ ，这一步骤最坏情况时间为 $T(3n/4)$

对于 $n < 75$ ，最坏情况时间为 $T(n) = O(1)$

2.9 线性时间选择

计算耗时分析

$$T(n) = O(n)$$

$T(n)$	SELECT(i, n)
$\Theta(n)$	{ 1. Divide the n elements into groups of 5. Find the median of each 5-element group by rote.
$T(n/5)$	{ 2. Recursively SELECT the median x of the $\lfloor n/5 \rfloor$ group medians to be the pivot.
$\Theta(n)$	3. Partition around the pivot x . Let $k = \text{rank}(x)$.
$T(3n/4)$	{ 4. if $i = k$ then return x elseif $i < k$ then recursively SELECT the i th smallest element in the lower part else recursively SELECT the $(i-k)$ th smallest element in the upper part

复杂度分析

$$T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \geq 75 \end{cases}$$

$$T(n) = O(n)$$

上述算法将每一组的大小定为5，并选取75作为是否作递归调用的分界点。这2点保证了 $T(n)$ 的递归式中2个自变量之和 $n/5 + 3n/4 = 19n/20 = \varepsilon n$ ， $0 < \varepsilon < 1$ 。这是使 $T(n) = O(n)$ 的关键之处。当然，除了5和75之外，还有其他选择。

补充两点：

- 实际上，SELECT算法运行缓慢，因为 n 前面的常数很大。
- 所以在很多实际应用中，**RandomizedSelect**随机算法比较实用。

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术;
 - (2) 大整数乘法;
 - (3) 棋盘覆盖;
 - (4) 合并排序和快速排序;
 - (5) 线性时间选择;
 - (6) 最接近点对问题 ;
 - (7) 循环赛日程表

2.10 最接近点对问题

计算几何学中研究的基本问题之一。

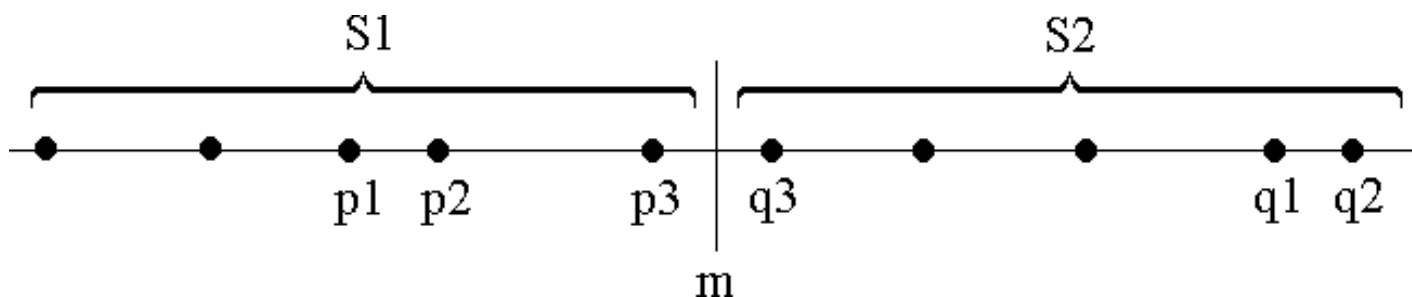
在涉及几何对象的问题中，常需要了解其邻域中其他几何对象的信息。例如，在空中交通控制问题中，若将飞机作为空间中移动的一个点来看待，则具有最大碰撞危险的两架飞机，就是这个空间中最接近的一对点。

最接近点问题：给定平面上的 n 个点，找其中的一对点，使得在 n 个点组成的所有点对中，该点距离最小

2.10 最接近点对问题

给定平面上 n 个点的集合 S ，找其中的一对点，使得在 n 个点组成的所有点对中，该点对间的距离最小。

为了使问题易于理解和分析，先来考虑**一维**的情形。此时， S 中的 n 个点退化为 x 轴上的 n 个实数 $x_1, x_2, \dots, x_n$ 。最接近点对即为这 n 个实数中相差最小的2个实数。

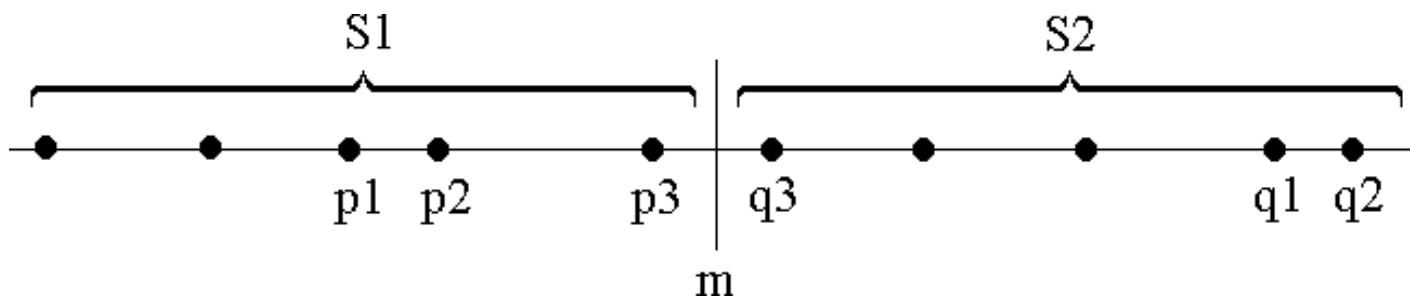


2.10 最接近点对问题

假设我们用x轴上某个点m将S划分为2个子集S1和S2，基于**平衡子问题**的思想，用S中各点坐标的**中位数**来作分割点。

递归地在S1和S2上找出其最接近点对 $\{p_1, p_2\}$ 和 $\{q_1, q_2\}$ ，并设 $d = \min\{|p_1 - p_2|, |q_1 - q_2|\}$ ，S中的最接近点对 $\{p_1, p_2\}$ 、或者是 $\{q_1, q_2\}$ ，或者是某个 $\{p_3, q_3\}$ ，其中 $p_3 \in S_1$ 且 $q_3 \in S_2$ 。

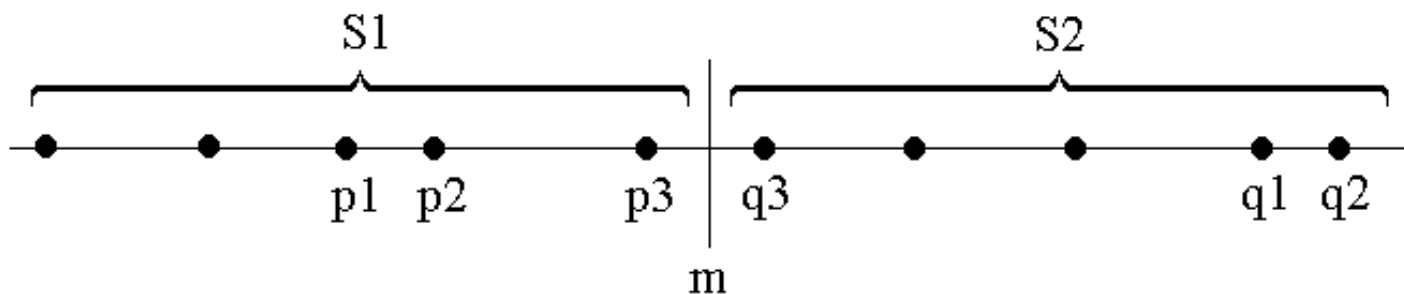
能否在线性时间内找到 p_3, q_3 ?



为什么不用均值?

2.10 最接近点对问题

- ◆如果S的最接近点对是 $\{p_3, q_3\}$ ，即 $|p_3 - q_3| < d$ ，则 p_3 和 q_3 两者与 m 的距离不超过 d ，即 $p_3 \in (m-d, m]$ ， $q_3 \in (m, m+d]$ 。
- ◆由于在 S_1 中，**每个长度为 d 的半闭区间至多包含一个点**(否则必有两点距离小于 d)，并且 m 是 S_1 和 S_2 的分割点，因此 $(m-d, m]$ 中至多包含 S 中的一个点。由图可以看出，**如果 $(m-d, m]$ 中有 S 中的点，则此点就是 S_1 中最大点**。
- ◆因此，我们用线性时间就能找到区间 $(m-d, m]$ 和 $(m, m+d]$ 中所有点，即 p_3 和 q_3 。从而我们用线性时间就可以将 S_1 的解和 S_2 的解合并成为 S 的解。



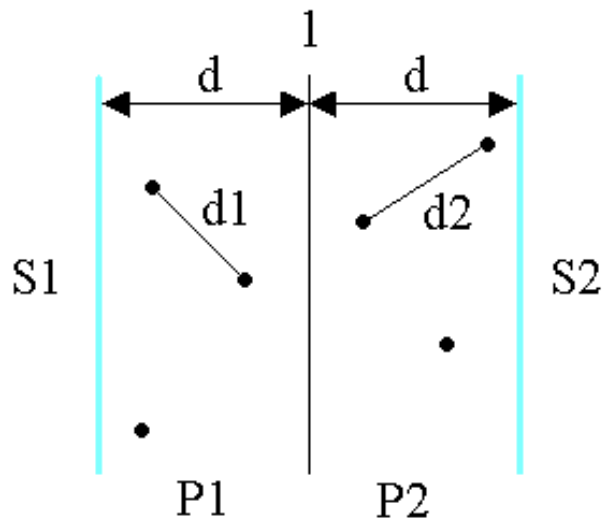
2.10 最接近点对问题

◆ 下面来考虑二维的情形。

➤ 选取一垂直线 $l: x=m$ 来作为分割直线。其中 m 为 S 中各点 x 坐标的中位数。由此将 S 分割为 S_1 和 S_2 。

➤ 递归地在 S_1 和 S_2 上找出其最小距离 d_1 和 d_2 ，并设 $d = \min\{d_1, d_2\}$ ， S 中的最接近点对或者是 d ，或者是某个 $\{p, q\}$ ，其中 $p \in S_1$ 且 $q \in S_2$ 。 P_1 表示 S_1 中距离 l 为 d 的区域

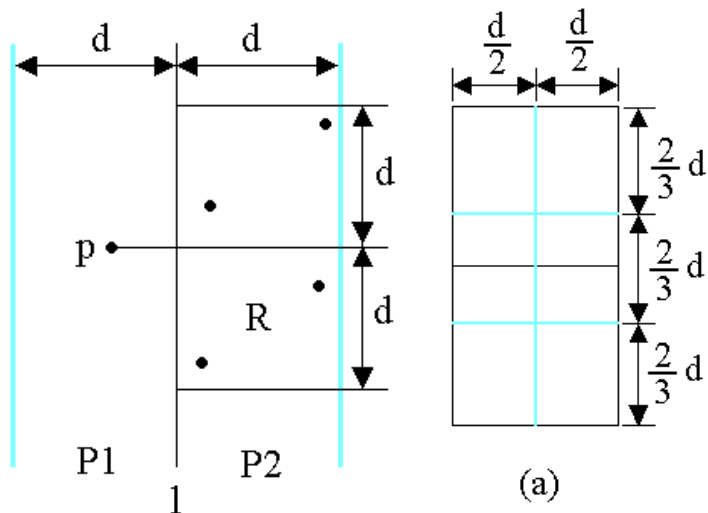
➤ 能否在线性时间内找到 p, q ?



2.10 最接近点对问题

能否在线性时间内找到 p, q ?

- ◆考虑 P_1 中任意一点 p ，它若与 P_2 中的点 q 构成最接近点对的候选者，则必有 $\text{distance}(p, q) < d$ 。满足这个条件的 P_2 中的点一定落在一个 $d \times 2d$ 的矩形 R 中
- ◆由 d 的意义可知， P_2 中任何2个 S 中的点的距离都不小于 d 。由此可以推出矩形 R 中最多只有6个 S 中的点。
- ◆因此，在分治法合并步骤中最多只需要检查 $6 \times n/2 = 3n$ 个候选者



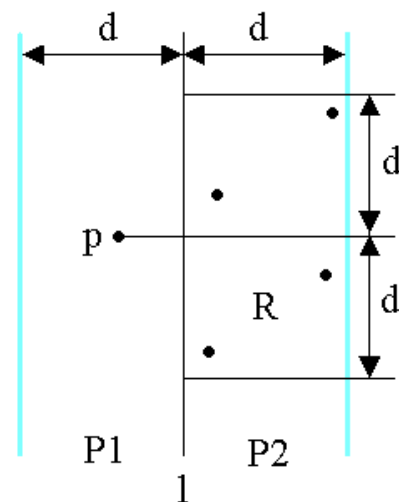
证明:将矩形 R 的长为 $2d$ 的边3等分，将它的长为 d 的边2等分，由此导出6个 $(d/2) \times (2d/3)$ 的矩形。若矩形 R 中有多于6个 S 中的点，则由鸽舍原理易知至少有一个 $(d/2) \times (2d/3)$ 的小矩形中有2个以上 S 中的点。设 u, v 是位于同一小矩形中的2个点，则

$$(x(u) - x(v))^2 + (y(u) - y(v))^2 \leq (d/2)^2 + (2d/3)^2 = \frac{25}{36}d^2$$

$\text{distance}(u, v) < d$ 。这与 d 的意义相矛盾。

2.10 最接近点对问题

- 为了确切知道检查哪6个点，可以将 p 和 P_2 中所有 S_2 的点投影到垂直线 l 上。由于能与 p 点一起构成最接近点对候选者的 S_2 中点一定在矩形 R 中，所以它们在直线 l 上的投影点距 p 在 l 上投影点的距离小于 d
- 因此，将 P_1 和 P_2 中所有 S 中点按其 y 坐标排好序，对 P_1 中所有点，对排好序的点列作一次扫描，就可以找出所有最接近点对的候选者。对 P_1 中每一点，最多只要检查 P_2 中6个点。



2.10 最接近点对问题

double **cpair2**(S)

{ n=|S|;

if (n < 2) return;

1、m=S中各点x间坐标的中位数·

//构造S1

S1={p∈S

S2={p∈S

2、d1=cpai

d2=cpair2(S2);

3、dm=min(d1,d2);

$O(1)$

4、P1是S1中距分割线l在dm之内所有点的集合;

P2是S2中距分割线l在dm之内所有点的集合;

将P1和P2中点依其y坐标值排序;

设X和Y是相应的已排好序的点列;

$O(n)$

复杂度分析

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

T(n)=O(nlogn)

距离;

6、d=min(dm,dl);

return d;

$O(1)$

}

有其
n)
扫

最小

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术;
 - (2) 大整数乘法;
 - (3) 棋盘覆盖;
 - (4) 合并排序和快速排序;
 - (5) 线性时间选择;
 - (6) 最接近点对问题 ;
 - (7) 循环赛日程表

2.11 循环赛日程表

有 $n=2^k$ 名选手，设计一个满足以下要求的比赛日程表：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) 循环赛一共进行 $n-1$ 天。

Day1	Day2	...	...	...	Day(n-3)	Day(n-2)	Day(n-1)
case1	case1				case1	case1	case1
case2	case2				case2	case2	case2
...	...				...	...	...

日程表直观设计如上，但为了便于计算，可以对数据结构进行简化

2.11 循环赛日程表

有 $n=2^k$ 名选手，设计一个满足以下要求的比赛日程表：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) 循环赛一共进行 $n-1$ 天。

将日程表设计成 n 行 n 列的表，在表中第 i 行、第 $j+1$ 列处填入第 i 个选手在第 j 天所遇到的对手（第一列是运动员编号）。

运动员编号	Day1	Day2	Day3	...	...	Day(n-2)	Day(n-1)
1							
2							
...							
n							

2.11 循环赛日程表

有 $n=2^k$ 名选手，设计一个满足要求的比赛日程表：

递归关系分析：

- 把 n 名选手分成两部分： $1 \sim n/2$ 和 $(n/2+1) \sim n$ ；
- $1 \sim n/2$ 日程表A：由数字 $1 \sim n/2$ 组成的 $(n/2) \times (n/2)$ 表格；
- $(n/2+1) \sim n$ 日程表B：由数字 $(n/2+1) \sim n$ 组成的 $(n/2) \times (n/2)$ 表格；
- n 名选手日程表是一个 $n \times n$ 的表格，把行和列分别分成两半，可以得到四个 $(n/2) \times (n/2)$ 的表格，记为L1, R1, L2, R2，其中L1与A完全一致，L2与B完全一致；

L1	R1
L2	R2

2.11 循环赛日程表

有 $n=2^k$ 名选手，设计一个满足要求的比赛日程表：

递归关系分析：

- R1是 $n/2+1 \sim n$ 组成的 $(n/2) \times (n/2)$ 表格，可以与L2完全一致，因此R2与L1完全一致；

递归终结状态（ $n=2$ 即 $k=1$ ）：

$n=2$ 时设编号分别为1，2，日程表如下：

L1	R1
L2	R2

1	2
2	1

2.11 循环赛日程表

设计一个满足以下要求的比赛日程表：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) 循环赛一共进行 $n-1$ 天。

按分治策略，将所有的选手分为两半， n 个选手的比赛日程表就可以通过为 $n/2$ 个选手设计的比赛日程表来决定。递归地用对选手进行分割，直到只剩下2个选手时，比赛日程表的制定就变得很简单。这时只要让这2个选手进行比赛就可以了。

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术;
 - (2) 大整数乘法;
 - (3) 棋盘覆盖;
 - (4) 合并排序和快速排序;
 - (5) 线性时间选择;
 - (6) 最接近点对问题 ;
 - (7) 循环赛日程表

分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 该问题的**规模缩小**到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的**相同问题**；
- 利用分解出的子问题的解**可以合并**为该问题的解；
- 该问题所**分解出的各个子问题是相互独立的**，即子问题之间不包含公共的子问题。

如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用动态规划较好。

作业

1. 最小的K个整数：输入整数数组arr，找出其中最小的k个数。例如，输入4、5、1、6、2、7、3、8这8个数字，则最小的4个数字是1、2、3、4。备注：使用线性时间选择方法完成

<https://leetcode-cn.com/problems/zui-xiao-de-kge-shu-lcof/>

2. 教材 算法分析2-7； 3. 算法分析2-15

4. 教材 算法实现2-6； 5. 算法实现2-11